

Plan: Fix Data Modeling & Graph Construction in DODGE

Context

The DODGE system (FastAPI + Neo4j + LLM pipeline) loads SAP Order-to-Cash JSONL data from 19 folders, builds a graph in Neo4j, and supports natural language queries. The current graph has critical structural problems:

- **Star clusters:** Wrong node identity (e.g., one node connected to hundreds)
- **Linear chains:** Item tables modeled as nodes create `Header → Item → Product` chains
- **Disconnected components:** Missing or incorrect relationships from Jaccard-based inference
- **Query failures:** No `DISTINCT` intent; multi-hop queries unreliable

Root cause: Bridge/item tables (`SalesOrderItems`, `DeliveryItems`, `InvoiceItems`) are modeled as separate nodes with composite keys, and relationships are inferred via value overlap heuristics instead of explicit `FK→PK` mappings.

Decisions (Confirmed)

- **8 CORE ENTITIES:** Original 6 + `Delivery` (`outbound_delivery_headers`) + `JournalEntry` (`journal_entry_items_accounts_receivable`)
- **Address → IGNORE:** Dropped from graph; can be added as node properties later if needed
- **Invoice -[BILLS]-> Delivery:** `referenceSdDocument` maps to `deliveryDocument` (will verify against data)
- **PLACED_ORDER uses businessPartner → soldToParty:** `FK→PK` principle, not the `customer` field

Files to Modify

#	File	Change
1	<code>backend/app/config/constants.py</code>	Add entity classification sets
2	<code>backend/app/modeling/graph_schema.py</code>	Extend <code>EdgeSchema</code> with bridge fields
3	<code>backend/app/graph/graph_builder.py</code>	Support bridge-table edges + rel properties
4	<code>backend/app/modeling/relationship_mapper.py</code>	<code>FK-only</code> edges + bridge table logic
5	<code>backend/app/ingestion/schema_inference.py</code>	No structural changes (kept for auto-inference fallback)
6	<code>backend/graph_schema_curated.json</code>	Rewrite: 8 nodes, bridge-based edges
7	<code>backend/app/llm/intent_extractor.py</code>	Add "distinct" to valid intents
8	<code>backend/app/llm/prompt_manager.py</code>	Update intent prompt (distinct + fix rank example)
9	<code>backend/templates/cypher/distinct.cypher</code>	NEW: distinct query template
10	<code>backend/templates/cypher/flow_gaps.cypher</code>	Update for new graph model
11	<code>backend/app/query/query_router.py</code>	Add distinct parameter mapping + auto-redirect
12	<code>backend/app/query/response_formatter.py</code>	Add distinct answer/explanation formatting
13	<code>backend/app/query/query_validator.py</code>	Add property-level validation (soft)

Step 1: Entity Classification (constants.py)

Add to backend/app/config/constants.py :

```
CORE_ENTITIES = {
    "business_partners",
    "sales_order_headers",
    "products",
    "plants",
    "billing_document_headers",
    "payments_accounts_receivable",
    "outbound_delivery_headers",
    "journal_entry_items_accounts_receivable",
}

BRIDGE_TABLES = {
    "sales_order_items",
    "outbound_delivery_items",
    "billing_document_items",
}

IGNORE_TABLES = {
    "product_descriptions",
    "business_partner_addresses",
    "customer_company_assignments",
    "customer_sales_area_assignments",
    "product_plants",
    "product_storage_locations",
    "sales_order_schedule_lines",
    "billing_document_cancellations",
}
```

Step 2: Extend EdgeSchema (graph_schema.py)

Add three optional fields to EdgeSchema :

```
class EdgeSchema(BaseModel):
    type: str
    from_node: str
    to_node: str
    join_on: tuple[str, str] # (source_field, target_id_field)
    source_entity: str | None = None # bridge table name (reads data from this DF)
    rel_properties: list[str] | None = None # columns to copy as relationship properties
    bridge_target_field: str | None = None # bridge column for target match (when !=
target id)
```

- source_entity : When set, GraphBuilder reads from this DataFrame instead of from_node's.
- bridge_target_field : When bridge column name differs from target node's id_field (e.g., material in bridge → product in Product node).

- All default to `None` → backward compatible with existing JSON schemas.

Also update `edge_descriptions()` to include `rel_properties` hints for LLM context.

Step 3: Update GraphBuilder (`graph_builder.py`)

Rewrite `merge_edges()` to handle bridge-table edges:

Key logic change:

1. When `edge.source_entity` is set → read from bridge table DataFrame
2. Source node matched by its `id_field` (column exists in bridge DF with same name)
3. Target node matched by `tgt_field` (target's id), value read from `bridge_target_field` column
4. When `edge.rel_properties` is set → add `SET r.prop = row.prop` for each property

Cypher output example (SalesOrderItems bridge):

```
UNWIND $rows AS row
MATCH (a:`SalesOrder` {`salesOrder`: row.`salesOrder`})
MATCH (b:`Product` {`product`: row.`material`})
MERGE (a)-[r:`CONTAINS`]->(b)
SET r.`requestedQuantity` = row.`requestedQuantity`, r.`netAmount` = row.`netAmount`
```

For non-bridge edges (direct FK): Behavior unchanged — reads from `from_node`'s DataFrame, no relationship properties.

Step 4: Replace Jaccard Edge Building (`relationship_mapper.py`)

4a. Filter nodes by CORE_ENTITIES in `build_schema()`

```
for entity_name, df in dataframes.items():
    if entity_name not in CORE_ENTITIES:
        continue
    # ... build node schema
```

4b. Replace `_build_edges()` with `_build_fk_edges()`

Same logic as current `_build_edges()` but:

- Both source and target entities must be in `CORE_ENTITIES`
- Keep `_fields_are_related()` semantic filter
- Keep `_GENERIC_FIELDS` blacklist
- Keep confidence threshold (candidates still come from `schema_inference` for auto-inference path)

4c. Add `_build_bridge_edges()` method

Hardcoded bridge table mappings:

Bridge Table	Edge	From → To	join_on	bridge_target_field
--------------	------	-----------	---------	---------------------

sales_order_items	CONTAINS	SalesOrder → Product	(salesOrder, product)	material
sales_order_items	PRODUCED_AT	SalesOrder → Plant	(salesOrder, plant)	productionPlant
outbound_delivery_items	FULFILLS	Delivery → SalesOrder	(deliveryDocument, salesOrder)	referenceSdDocument
outbound_delivery_items	SHIPS_FROM	Delivery → Plant	(deliveryDocument, plant)	plant
billing_document_items	INVOICES	Invoice → Product	(billingDocument, product)	material
billing_document_items	BILLS	Invoice → Delivery	(billingDocument, deliveryDocument)	referenceSdDocument

Step 5: Update Curated Schema (graph_schema_curated.json)

New Node List (8 core entities, no item nodes):

Label	source_entity	id_field
Customer	business_partners	businessPartner
SalesOrder	sales_order_headers	salesOrder
Product	products	product
Plant	plants	plant
Delivery	outbound_delivery_headers	deliveryDocument
Invoice	billing_document_headers	billingDocument
JournalEntry	journal_entry_items_accounts_receivable	[companyCode, fiscalYear, accountingDocument, accountingDocumentItem]
Payment	payments_accounts_receivable	[companyCode, fiscalYear, accountingDocument, accountingDocumentItem]

Removed: SalesOrderItem, DeliveryItem, InvoiceItem, Address

New Edge List:

Direct FK edges (no bridge):

- Customer -[PLACED_ORDER]-> SalesOrder (businessPartner → soldToParty)
- Invoice -[BILLED_TO]-> Customer (soldToParty → businessPartner)
- Invoice -[GENERATES]-> JournalEntry (accountingDocument → accountingDocument)
- JournalEntry -[CLEARED_BY]-> Payment (clearingAccountingDocument → accountingDocument)

5. Payment -[PAID_BY]-> Customer (customer → businessPartner)

Bridge-table edges (with source_entity + rel_properties): 6-11. See table in Step 4c above.

Step 6: Verify Node-Edge Separation

`build_all()` already runs nodes-first, edges-second. **No code change needed.** Just verify during testing.

Step 7: Add DISTINCT Intent (6 sub-changes)

7a. `intent_extractor.py`

Add "distinct" to `VALID_INTENT_TYPES` set.

7b. `prompt_manager.py`

Add to `intent_extraction` prompt:

- `distinct` in the `intent_type` list
- New bullet: "- distinct: List unique/distinct values of a property. Trigger: 'types', 'kinds', 'different', 'unique', 'categories'. Set filters.property_name."
- Fix rank example to reference new graph model (no InvoiceItem)

7c. `templates/cypher/distinct.cypher` (NEW FILE)

```
-- name: distinct
-- description: List distinct values of a specific property on nodes of a given label
-- params: label, property_name
MATCH (n:`${label}`)
WHERE n.`${property_name}` IS NOT NULL
RETURN DISTINCT n.`${property_name}` AS value, count(*) AS occurrences
ORDER BY occurrences DESC
LIMIT 100
```

7d. `query_router.py`

- Add `_DISTINCT_KEYWORDS = {"types", "kinds", "different", "unique", "categories", "distinct"}`
- Add auto-redirect: if intent is `find_node` / `list_neighbors` but query contains distinct keywords → redirect to `distinct`
- Add `elif template.name == "distinct" in _map_parameters()`:

```
params["label"] = entities[0] if entities else ""
params["property_name"] = str(filters.get("property_name", ""))
```

7e. `response_formatter.py`

Add distinct handling in both `_build_answer()` and `_build_explanation()`:

- **Answer:** Compute values in backend: "Found {count} distinct {prop} values: {val1}, {val2}, ..."

- **Explanation:** "Listing distinct values of {prop} across all {label} nodes."
- Do NOT rely on LLM for counting — use `len(data)` directly.

7f. No change to `query_validator.py` for distinct (labels already validated).

Step 8: Fix Response Formatting for DISTINCT

Covered in Step 7e. Key requirement: the `_build_answer()` path (not LLM) must handle distinct results by extracting values from the `data` list and computing count programmatically.

Step 9: Property Validation (`query_validator.py`)

Add `_extract_variable_labels()` helper to map Cypher variables to node labels. Then in `validate()` :

- Extract `(variable, property)` pairs from Cypher
 - Look up variable → label binding
 - Check property exists in node's schema
 - **Soft warning only** (`logger.debug()`), NOT hard error — relationship properties and Cypher functions can look like property access
-

Step 10: Rebuild Graph (Operational)

After all code changes:

1. `DELETE /api/v1/graph/clear`
 2. `POST /api/v1/graph/build` with updated curated schema, `clear_existing: true`
 3. Verify with `GET /api/v1/graph/status`
 4. Visual verification with `GET /api/v1/graph/explore`
-

Step 11: Update `flow_gaps.cypher` Template

Rewrite to use new direct relationships (no item node hops):

```
MATCH (so:`SalesOrder`)
OPTIONAL MATCH (so)-[:FULFILLS]-(d:`Delivery`)
OPTIONAL MATCH (inv:`Invoice`)-[:BILLS]->(d)
OPTIONAL MATCH (inv)-[:GENERATES]->(je:`JournalEntry`)
OPTIONAL MATCH (je)-[:CLEARED_BY]->(pay:`Payment`)
WITH so,
    count(DISTINCT d) > 0 AS has_delivery,
    count(DISTINCT inv) > 0 AS has_invoice,
    count(DISTINCT je) > 0 AS has_journal_entry,
    count(DISTINCT pay) > 0 AS has_payment
WHERE NOT (has_delivery AND has_invoice AND has_journal_entry AND has_payment)
RETURN so.`salesOrder` AS sales_order, has_delivery, has_invoice, has_journal_entry,
has_payment,
CASE
    WHEN NOT has_delivery THEN 'No delivery'
    WHEN has_delivery AND NOT has_invoice THEN 'Delivered but not billed'
    WHEN has_invoice AND NOT has_journal_entry THEN 'Billed but no journal entry'
```

```
    WHEN has_journal_entry AND NOT has_payment THEN 'No payment'
    ELSE 'Incomplete flow'
END AS gap_type
ORDER BY gap_type
LIMIT 50
```

Verification Plan

1. **Schema validation:** Load updated curated JSON → verify 8 nodes, 11 edges, no item nodes
2. **Graph build:** Run full ingestion → verify node counts (no SalesOrderItem/DeliveryItem/InvoiceItem nodes)
3. **Graph structure:** GET /api/v1/graph/explore → verify no star clusters, no long chains
4. **Bridge relationships:** Verify CONTAINS edges have properties (qty, price)
5. **Distinct query:** Ask "Show different types of plants" → verify DISTINCT template used, correct results
6. **Multi-hop query:** Ask "Which customer has the most orders" → verify traversal works
7. **Flow gaps:** Ask "Show incomplete flows" → verify updated template returns results
8. **Negative test:** Ask about a non-existent label → verify rejection